

JMeter — alignment with this project (server-v2)

本机路径：文中示例里的 `C:\jmeter`、`C:\tools\apache-jmeter-5.6.3`、`D:\6650\assignment 3` 等请一律改成你电脑上的实际路径。JMeter 根目录指含有 `bin\jmeter.bat` 的那一层；也可设环境变量 `JMETER_HOME` 指向该目录，再用 `$env:JMETER_HOME\bin\jmeter.bat`。

Course docs often use placeholder paths (`/api/messages`, `/chat` without room). **This stack uses:**

Kind	URL	Notes
WebSocket	<code>ws://<host>:8080/chat/{roomId}</code>	<code>roomId</code> = 1–20 (see <code>ChatWebSocketHandler</code>). Example: <code>ws://localhost:8080/chat/5</code>
HTTP	<code>http://<host>:8080/health</code>	JSON health — good for load / baseline without WS plugin
HTTP	<code>http://<host>:8080/metrics</code>	Heavy; avoid <code>refreshMaterializedViews=true</code> through ALB (504 risk). Use for light checks or after load ends

ALB: use `http://<alb-dns>/health` (port **80** → target 8080). WebSocket from JMeter: `ws://<alb-dns>/chat/1` (no TLS in typical class setup; use `wss` only if you terminate TLS on ALB).

走 ALB 跑 HTTP 计划 (推荐与线上一致)

本仓库里的 `assignment-*.jmx` / `health-baseline.jmx` 已用 `${__P(host,localhost)}` 和 `${__P(port,8080)}`。经 **ALB** 时必须改为 主机 = **ALB DNS** (仅域名)、端口 = **80** (不要写 8080)。

方式 **A** — 脚本 (自动读 `terraform output alb_dns_name`)

参数名**不要用** `-Jmx` / `-JmeterHome` (PowerShell 易和 JMeter 的 `-J` 混淆)。请用 `-TestPlan`、`-JMeterInstall` (JMeter 解压根目录，内含 `bin\jmeter.bat`)。不设 `-JMeterInstall` 时会依次试 `JMETER_HOME`、where `jmeter.bat`、常见路径。

```
cd "D:\6650\assignment 3\load-tests\jmeter" # 改成你本机仓库路径
.\run-http-via-alb.ps1 -TestPlan health-baseline.jmx
# 若脚本找不到 jmeter.bat，指定你的 JMeter 根目录 ( 勿照抄示例路径 )：
.\run-http-via-alb.ps1 -TestPlan health-baseline.jmx -JMeterInstall "你的JMeter根目录"
.\run-http-via-alb.ps1 -TestPlan assignment-baseline-100k-5min.jmx -JMeterInstall "你的JMeter根目录"
```

脚本若发现同目录下的 `jmeter-run.properties`，会自动 `-q` 加载 (HttpClient4 连接池/保活等)，并为 CLI 设置适度堆内存；可用 `-SkipRunProperties` 关闭。高并发压测仍建议在 **Linux/WSL** 或 分布式 **JMeter** 上执行，以免 Windows 本机出站端口成为瓶颈。

同区域 EC2 上跑 JMeter (推荐：与 ALB 同 region，减少 TCP 建连失败)

在 `deployment/terraform/terraform.tfvars` 中设置 `enable_jmeter_ec2 = true` (需 `enable_eks = false` 且 `enable_alb = true`) , `terraform apply` 后会多一台 **Amazon Linux 2023**, `user-data` 已安装 **Java 17** 与 **Apache JMeter** (默认 **5.6.3**) 到 `/opt/apache-jmeter`。

1. `terraform apply` 成功后 : `terraform output jmeter_public_ip`、`terraform output alb_dns_name`。
2. 上传计划文件 (本机 **PowerShell**) : 在 `load-tests/jmeter` 执行 `.\upload-plans-to-jmeter-ec2.ps1 -KeyPath "<与 Terraform 密钥成对的私钥路径>"` (脚本会 `terraform output` 读 IP 并 `scp` 主要 `.jmx` 与 `jmeter-run.properties` 到 `~/jmeter-plan/`) 。
3. SSH : `ssh -i <私钥> ec2-user@<IP>` , 在 EC2 上 (**Linux** 用 `jmeter` 不是 `jmeter.bat`) :

```
source /etc/profile.d/jmeter.sh
cd ~/jmeter-plan
# 在笔记本上执行 terraform output -raw alb_dns_name , 把结果贴到下面 :
export ALB="cs6650-chat-xxxx.us-east-1.elb.amazonaws.com"
/opt/apache-jmeter/bin/jmeter -n -t assignment-stress-30min.jmx \
  -q jmeter-run.properties \
  -Jhost="$ALB" -Jport=80 \
  -l ~/jmeter-results/stress.jtl -e -o ~/jmeter-results/stress-report
```

4. 用 `scp` 把 `~/jmeter-results/` 拉回笔记本写报告。

实例类型默认 `t3.large` (可在 `terraform.tfvars` 里改 `instance_type_jmeter`) 。详见 `deployment/terraform/README-EC2-LAB.md` 中的 JMeter 小节。

若 `terraform output jmeter_public_ip` 报 “**Output not found**” : 说明当前目录下的 Terraform 配置里还没有这个 output (常见 : 代码未更新、不在 `deployment/terraform` 下执行) 。请 `git pull` 拉取含 `jmeter.tf` 与 `outputs.tf` 里 `jmeter_*` 的提交 , 在该目录执行 `terraform init` , 再 `terraform output` 查看是否出现 `jmeter_public_ip`。

方式 B — 命令行 -J

```
$JMETER_ROOT = "你的JMeter根目录" # 含 bin\jmeter.bat , 勿照抄
$REPO = "你的assignment3仓库根目录" # 含 load-tests\jmeter
$alb = terraform -chdir="$REPO\deployment\terraform" output -raw alb_dns_name
& "$JMETER_ROOT\bin\jmeter.bat" -n -t "$REPO\load-tests\jmeter\assignment-
baseline-100k-5min.jmx" `
  -Jhost="$alb" -Jport=80 -l "$REPO\load-tests\jmeter\results\run.jtl" -e -o
"$REPO\load-tests\jmeter\results\run-report"
```

方式 C — 属性文件 : 复制 `jmeter-alb.properties.example` → `jmeter-alb.properties` , 把 `host=` 改成你的 ALB DNS, `port=80` , 然后 :

`jmeter -q -p jmeter-alb.properties -n -t ...` (仍需命令行指定 `-t`、`-l` 等) 。

WebSocket 经 ALB : **Server / 主机** = 同上 ALB DNS ; **Port** = **80** ; **Path** = `/chat/1.../chat/20` ; **Protocol** = **ws** (当前 ALB 为 HTTP 80、无证书时用 **ws** , 不是 **wss**) 。

Install (summary)

- **JMeter 5.6+, JDK 8+** (JDK 17+ matches this project).
- Windows：解压到任意目录，该目录即 **JMeter 根目录**（下有 `bin\jmeter.bat`）；Linux 例：`/opt/apache-jmeter-5.6.x`。
- **GUI:** `bin\jmeter.bat` (Windows) or `bin/jmeter`
- **CLI:**
`bin/jmeter -n -t plan.jmx -l results.jtl -e -o report_html`

WebSocket plugin (if you load-test WS in JMeter): Plugins Manager → *WebSocket Samplers by Peter Doornbosch* (or manual JAR into `lib/ext`). See your course handout.

Running tests (course checklist)

GUI mode (editing / debugging only):

- Linux / macOS: `./bin/jmeter`
- Windows: `bin\jmeter.bat` (or run from `bin` as `.\jmeter.bat`)

Non-GUI mode (real load runs):

```
./bin/jmeter -n -t your-test-plan.jmx -l results.jtl -e -o report-folder
```

Generate HTML report from an existing JTL (no re-run):

```
./bin/jmeter -g results.jtl -o html-report-folder
```

On Windows, use `.\jmeter.bat` instead of `./bin/jmeter`, and use quoted paths if directories contain spaces. Example **baseline / stress** commands for this repo are in **Course test plans** later in this file.

Test scenarios (rubric) → this repo

Use your assignment’s scenarios; map them to the files below and **state the mapping in your PDF**.

Scenario	Rubric (typical)	In this repo
Baseline	~ 1000 concurrent users; ~ 100K API calls; ~ 5 min ; 70% reads / 30% writes ; report response time, throughput, error rate	<code>assignment-baseline-100k-5min.jmx</code> : 700+300 threads (70/30 split), 100K HTTP samples total, default 270 s ramp (<code>ramp_sec</code> ; override e.g. 240 / 300). Note: duration is driven by loops reaching 100K samples, not a fixed 300 s wall-clock timer — if the grader requires a strict 5-minute duration , add a Scheduler duration and tune throughput in a copy of the plan (see baseline subsection below).

Scenario	Rubric (typical)	In this repo
Stress	500 concurrent users; 200K–500K calls over 30 min; mixed read/write; observe breaking point and resource utilization	assignment-stress-30min.jmx only (graded scenario): 350+150 threads, 30 min, default caps 11667+5000 samples/min per thread group (Constant Throughput) → ~500K HTTP / 30 min, 270s ramp, metrics TG delayed 120s. Stress is slower req/s than baseline by design. Do not substitute ad-hoc lower thread/cap copies for the submission unless your handout explicitly allows it.

Operations (read/write): rubric “writes” are approximated as GET /metrics (heavy); “reads” as GET /health. Literal chat traffic is WebSocket — add the WebSocket plan from WebSocket Sampler if you must show wire-format chat in JMeter.

HTTP-only test plan (no plugin) — quick baseline

1. **Test Plan** → **Thread Group**
 - Threads, ramp-up, duration or loop count per your assignment.
2. **HTTP Request Defaults**
 - Server: localhost or **ALB DNS only** (no http://)
 - Port: 8080 (直连 server) 或 80 (经 ALB · 与上面「走 ALB」一致)
3. **HTTP Request** sampler
 - Path: /health
 - Method: GET
4. **Listeners:** Summary Report, Aggregate Report; optional **Save Response to file** off for load runs.
5. **Assertions:** Response code 200; body contains "UP" or similar if you want stricter checks.

Use this for **before/after optimization** comparisons (throughput, p95/p99 as reported by JMeter for HTTP). This measures **ingress + health stack**, not full chat persistence latency.

WebSocket Sampler (course handout + this project)

Install **WebSocket Samplers by Peter Doornbosch** (Plugins Manager → Available Plugins → search WebSocket → install → restart JMeter). After restart, samplers appear under **Sampler** (names may be WebSocket Open Connection, WebSocket Single Write, etc., depending on version—not always the literal words “WebSocket Sampler”).

Course PDF — WebSocket testing (what matches, what to change)

Assignment handouts usually include **plugin install** (two methods), **connection settings**, a **small test plan tree**, and **troubleshooting**. The install steps are the same here:

Method	Action
1 — Plugins Manager (recommended)	Put <code>jmeter-plugins-manager-*.jar</code> in <code>lib/ext</code> , restart JMeter → Options → Plugins Manager → Available Plugins → install WebSocket Samplers by Peter Doornbosch → apply and restart.

Method	Action
2 — Manual	Release JAR(s) from ptrd/jmeter-websocket-samplers : main plugin in lib/ext , any dependency JARs in lib/ , restart JMeter.
WebSocket test configuration (PDF vs this codebase): follow your PDF for <i>timeouts</i> and “ <i>use existing connection</i> ” only after a connection is opened. Replace path , port , and JSON as follows — the PDF’s /chat + {"userId":"user123",...,"roomId":"room1"} example does not work on server-v2 .	
PDF / handout	server-v2
Server:	Same, or ALB DNS (host only, no http:// / ws:// in the server field — depends on localhost sampler UI).
Port: 8080	8080 when hitting the app directly; 80 when using ALB (HTTP listener → target 8080).
Path: /chat	/chat/1 ... /chat/20 (numeric room in the path). Plain /chat is wrong here.
Protocol: ws / wss	ws for class HTTP:80 or localhost:8080 without TLS; wss only if the front end terminates TLS.
Connection / response timeout	PDF values (e.g. 20000 / 6000 ms) are a starting point; for TEXT under load, increase read/response timeout if you see failures (6000 ms is often tight).
Add → WebSocket Sampler	With this plugin you typically add WebSocket Open Connection , then WebSocket Single Write (JOIN + loop TEXT), then WebSocket Close — not a single ambiguous sampler for the whole flow.
Use existing connection	Checked on Single Write samplers after Open Connection (first write after open: follow plugin behavior — some UIs need it unchecked on the very first frame).
Request JSON (short example)	Must include messageId , numeric userId , alphanumeric username (3–64), non-empty message , ISO timestamp (yyyy-MM-ddTHH:mm:ss, length ≥ 19), messageType : JOIN then TEXT . See templates below.

PDF test plan shape (100 users → Open → loop writes → Close) is correct; swap in **path** **/chat/{1–20}** and the **full JSON** from the next subsection.

Handout “WebSocket Test Plan Example” — same shape, **server-v2** settings

Course tree (conceptually):

```
Thread Group (100 users)
├── WebSocket Open Connection
├── Loop Controller (10 iterations)
│   └── WebSocket Single Write
└── WebSocket Close Connection
```

Adapted for this repo (room in path, valid JSON, JOIN before TEXT):

100 users = 100 unique `userId` values (no CSV required): use `${__threadNum}` (starts at 1 per thread) plus a fixed offset so each thread gets a **stable, distinct** numeric id. Use the **same** expression for JOIN and every TEXT in that thread so the virtual user does not “change identity” mid-run.

- **`userId`** (digits only, range **1–100000**): `"${__intSum(${__threadNum},10000,)}"` → thread 1 → `10001`, thread 100 → `10100`.
- **`username`** (3–64 chars, letters and digits only): `"user${__intSum(${__threadNum},10000,)}"` → `user10001` ... `user10100`.

If you ever run more than **90,000** threads with this offset, lower the offset or switch to CSV so `userId` stays ≤ 100000.

```
Thread Group (100 users, ramp e.g. 60s)
├─ WebSocket Open Connection
│   └─ Server: localhost, Port: 8080, Path: /chat/1    ← use /chat/1 ... /chat/20,
NOT /chat alone
│   └─ Protocol: ws, timeouts per handout (increase read timeout if TEXT fails
under load)
├─ WebSocket Single Write – JOIN (one frame before the loop)
│   └─ Message:
│       {"messageId":"${__UUID}","userId":"${__intSum(${__threadNum},10000,)}",
│        "username":"user${__intSum(${__threadNum},10000,)}","message":"Hello",
│        "timestamp":"${__time(yyyy-MM-dd'T'HH:mm:ss'Z')}","messageType":"JOIN"}
├─ Loop Controller (10 iterations)
│   └─ WebSocket Single Write
│       └─ Use existing connection: ✓
│       └─ Message:
│           {"messageId":"${__UUID}","userId":"${__intSum(${__threadNum},10000,)}",
│            "username":"user${__intSum(${__threadNum},10000,)}",
│            "message":"Test message ${__counter(FALSE,)}",
│            "timestamp":"${__time(yyyy-MM-
dd'T'HH:mm:ss'Z')}","messageType":"TEXT"}
└─ WebSocket Close Connection
```

Alternative: CSV Data Set Config with columns `userId`, `username` (e.g. 100 rows) and reference `"${userId}" / "${username}"` in the JSON when you need ids to match a real account table.

The handout’s minimal line `{"userId": "${userId}", "message": "Test message ${__counter()}"}` **is not enough** for `server-v2` — you still need `username`, `timestamp`, `messageType`, and a **numeric** `userId` (see table below).

Troubleshooting (handout + this app)

Issue	What to check
Connection refused	<code>server-v2</code> on 8080 , Docker/Rabbit/Postgres up; <code>check-stack.ps1</code> .
Plugin not found	JAR in <code>lib/ext</code> , restart JMeter ; sampler names vary by version.

Issue	What to check
Timeout	Raise connection/read timeouts; <code>/metrics</code> unrelated — WS is separate.
Message format	Match <code>ChatWebSocketHandler</code> : path room 1–20 , full JSON, JOIN (recommended) then TEXT .

1. What to change vs the generic handout

Handout example	For server-v2 you must use
Path: <code>/chat</code>	<code>/chat/{roomId}</code> with <code>roomId</code> = 1–20 (integer), e.g. <code>/chat/1</code> , <code>/chat/5</code> . Plain <code>/chat</code> is wrong for this codebase.
Example JSON with <code>"roomId": "room1"</code>	Room is taken from the URL path , not from a <code>room1</code> -style id. JSON still needs valid fields below; you can omit <code>roomId</code> in the body or use <code>"5"</code> —the server validates path room.
<code>"userId": "user123"</code>	<code>userId</code> must be digits only , range 1–100000 (e.g. <code>"12345"</code>).
<code>"username": "user123"</code>	3–64 chars, letters and digits only (e.g. <code>"user12345"</code>).

2. Suggested GUI layout (one thread = one chat user)

1. **Thread Group** — set users, ramp-up, loops as needed. **Each thread = one distinct chat user** — use `userId` / `username` expressions from the section above (or CSV) so 100 threads ⇒ 100 different ids.
2. **WebSocket Open Connection** (or your plugin’s open sampler):
 - **Server:** `localhost` (host only, no `http://`).
 - **Port:** `8080`.
 - **Path:** `/chat/1` (or `/chat/${__Random(1,20)}` if the sampler supports expressions).
 - **Protocol:** `ws` (use `wss` only if TLS terminates in front).
 - **Connection timeout:** `20000` ms (or as handout).
 - **Response / read timeout:** start with ≥ **10000 ms** for TEXT under load (`6000` may be tight).
3. **WebSocket Single Write** — **first** frame (do **not** use “existing connection” on the very first write if your plugin ties that to an already-open socket):
 - **Request data (JOIN):**

```
{ "messageId": "${__UUID}", "userId": "${__intSum($__threadNum,10000,)}", "username": "user${__intSum($__threadNum,10000,)}", "message": "Hello", "timestamp": "${__time(yyy-MM-dd'T'HH:mm:ss'Z')}", "messageType": "JOIN" }
```

4. **Loop Controller** (e.g. 10 iterations) → **WebSocket Single Write** for **TEXT**:
 - Check **Use existing connection** (per handout) so frames reuse the same socket.
 - **Request data:**


```
{
  "messageId": "${__UUID}",
  "userId": "${__intSum(__threadNum, 10000, )}",
  "username": "user${__intSum(__threadNum, 10000, )}",
  "message": "Test ${__counter(FALSE, )}",
  "timestamp": "${__time(yyyy-MM-dd'T'HH:mm:ss'Z')}",
  "messageType": "TEXT"
}
```

- Each TEXT should use a **new messageId** (`${__UUID}`) for idempotency, matching `client_part2`.
- 5. Optional **WebSocket Single Write** — **LEAVE** with same `userId` / `username` / `timestamp` rules, `messageType": "LEAVE"`.
- 6. **WebSocket Close** (if your plugin provides it) — clean shutdown.

Field rules (server ChatWebSocketHandler): `message` non-empty; `timestamp` like ISO `yyyy-MM-ddTHH:mm:ss` with `T` and colons (length ≥ 19); `messageType` one of **JOIN**, **TEXT**, **LEAVE**.

3. Why this is not the handout’s one-liner JSON

Handout sample `{ "userId": "user123", "message": "Hello", "roomId": "room1" }` **fails** this server: `userId` must be numeric, `username` is required, `timestamp` and `messageType` are required, and room must be `/chat/1.../chat/20`, not `room1`.

4. Practical note (submission)

Many teams still use **HTTP** plans (`assignment-baseline-100k-5min.jmx`) for rubric throughput tables and add a **smaller** WebSocket plan (screenshots) to show literal **JOIN/TEXT** traffic. Document both if TA asks.

Course test plans (Assignment 3 PDF / rubric)

Approach A (implemented in this repo): use **two Thread Groups** and set **thread counts** so they match roughly **70% / 30%** of request volume (baseline: 700+300; stress: 350+150). Inside each group, keep the usual **HTTP Request Defaults** → **HTTP Request** → **assertions**; you do **not** need a single group with Throughput Timer or If Controller + random split.

Note: `health-baseline.jmx` has only **one** Thread Group — it is a small smoke test, not the full baseline scenario from the rubric.

Handouts often cite `/api/messages` and `/chat` without a room id. **This repo’s JMeter plans use real paths** and map **70% read / 30% write** as follows (say this explicitly in your PDF):

Handout	In these .jmx files
70% reads	<code>GET /health</code> (light HTTP)
30% writes	<code>GET /metrics</code> without <code>refreshMaterializedViews</code> (heavy read/analytics; stresses DB + cache). <i>True</i> chat writes are WebSocket — add the WebSocket plugin + <code>client_part2</code> -style JSON if the grader insists on wire-format “writes”. Note: on Kubernetes (<code>k8s/server.yaml</code>), <code>server.metrics.allow-http-mv-refresh=false</code> — requests with <code>refreshMaterializedViews=true</code> return 403 ; keep JMeter on plain <code>/metrics</code> .

Baseline scenario (~1000 users, ~100K samples, 5 min)

- **File:** `assignment-baseline-100k-5min.jmx` — **700** threads × **100** loops on `/health` + **300** × **100** on `/metrics` = **100,000** HTTP samples; default ramp **270 s** (override with `-Jramp_sec=`, e.g. `240` or `300`).
- **Rubric “5 minutes”:** this plan matches **100K samples** and **~1000 threads** quickly; wall-clock time may be **shorter or longer** than 300 s depending on SUT speed. For a **strict 300 s** run, duplicate the plan in the GUI, set the Thread Group **Duration** to **300** seconds, switch loops to **forever** (or high), and use **Constant Throughput Timer** (or similar) so **70/30** traffic sums to **~100K** requests over 5 minutes — verify sample counts in the HTML report.
- **Before running:** start Docker stack, **server-v2** (:8080), **consumer-v3** (:8081). Run `database/check-stack.ps1` if unsure.

Non-GUI (Windows) : 先把 `JMETER_ROOT`、`REPO` 换成你本机路径。

```
$JMETER_ROOT = "你的JMeter根目录"
$REPO = "你的assignment3仓库根目录"
cd "$JMETER_ROOT\bin"
.\jmeter.bat -n -t "$REPO\load-tests\jmeter\assignment-baseline-100k-5min.jmx" `
-l "$REPO\load-tests\jmeter\results\baseline-100k.jtl" `
-e -o "$REPO\load-tests\jmeter\results\baseline-100k-report"
```

Optional overrides: `-Jhost=localhost -Jport=8080 -Jread_threads=700 -Jwrite_threads=300 -Jread_loops=100 -Jwrite_loops=100 -Jramp_sec=270`

500,000 HTTP samples (仍约 70/30) : 线程数仍为 700+300 时，令 `read_loops` 与 `write_loops` 均为 **500** ($700 \times 500 + 300 \times 500 = 500000$)。经 ALB：`.\run-http-via-alb.ps1 -TestPlan assignment-baseline-100k-5min.jmx -ReadLoops 500` (脚本会将 `WriteLoops` 同步为 500)；本机 JMeter 可传 `-Jread_loops=500 -Jwrite_loops=500`。

(可选) Baseline 同款、固定 30 分钟墙钟 — `assignment-baseline-30min-duration.jmx`

不是 rubric 表里的「Stress」行 课程要求的 **30 分钟 / 500 用户 / 200K–500K** 场景请用上表中的 `assignment-stress-30min.jmx`。本文件仅供你想做「与 100k baseline 同线程配比、但墙钟跑满 30 分钟」的额外实验。

- **File:** `assignment-baseline-30min-duration.jmx` — **700+300**，无限循环 + **Scheduler duration** (默认 **1800 s**)，无 Constant Throughput；默认 **270 s** ramp、**metrics** 与 **health** 同时起步 (**delay 0**)，与 `assignment-baseline-100k-5min.jmx` 一致。总 HTTP 次数不固定。

经 ALB :

```
.\run-http-via-alb.ps1 -TestPlan assignment-baseline-30min-duration.jmx `
-JMeterInstall "你的JMeter根目录" `
-Port 80
```

本机 JMeter :

```
.\jmeter.bat -n -t "$REPO\load-tests\jmeter\assignment-baseline-30min-
duration.jmx" `
-Jhost=<host> -Jport=<port> `
-l "$REPO\load-tests\jmeter\results\baseline-30min.jtl" `
-e -o "$REPO\load-tests\jmeter\results\baseline-30min-report"
```

Stress scenario (500 users, 30 min, ~500K samples @ default caps)

- **File:** `assignment-stress-30min.jmx` — **350 + 150** threads, **30 min**, **Constant Throughput Timer** per group (**11667 + 5000** samples/min → ~**500K** total over 30 min, rubric **200K–500K** upper band). Timer **calcMode 2** = *All active threads in current thread group* (shared cap per group, not per thread). **Ramp 270 s** (default); **metrics thread group starts 120 s later** than the health group. **Why slower than baseline?** Baseline has **no** throughput cap—threads race to finish 100K loops; Stress **paces** to a sustained target over **30 minutes**.
- **Troubleshooting only (changes ramp/delay vs rubric defaults):** if you see many `HttpHostConnectException` / `getsockopt` from your **laptop** to the ALB, try **wired Ethernet**, run JMeter from a **same-region EC2**, or use `.\run-http-via-alb.ps1 -StressWindows` (script injects **900 s** ramp and **180 s** metrics delay — **disclose in the PDF** if you use this; it is **not** the default `assignment-stress-30min.jmx`). **Zero** TCP errors cannot be guaranteed from every home ISP.
- To reach ~**500K** samples, edit the two **Constant Throughput Timer** literals in `assignment-stress-30min.jmx` only if your **assignment handout** allows tuning; otherwise keep defaults.

```
$JMETER_ROOT = "你的JMeter根目录"
$REPO = "你的assignment3仓库根目录"
cd "$JMETER_ROOT\bin"
.\jmeter.bat -n -t "$REPO\load-tests\jmeter\assignment-stress-30min.jmx" `
-l "$REPO\load-tests\jmeter\results\stress-30m.jtl" `
-e -o "$REPO\load-tests\jmeter\results\stress-30m-report"
```

Optional: `-Jduration_sec=1800, -Jramp_sec=270, -Jwrite_group_startup_delay_sec=120` (defaults match the `.jmx`). **ALB** (与作业默认一致) : `.\run-http-via-alb.ps1 -TestPlan assignment-stress-30min.jmx -JMeterInstall "<你的 JMeter 根目录>"`。

Non-GUI — small smoke test (`health-baseline.jmx`)

```
$JMETER_ROOT = "你的JMeter根目录"
$REPO = "你的assignment3仓库根目录"
cd "$JMETER_ROOT\bin"
.\jmeter.bat -n -t "$REPO\load-tests\jmeter\health-baseline.jmx" -l "$REPO\load-
tests\jmeter\results\smoke.jtl" -e -o "$REPO\load-tests\jmeter\results\smoke-
report"
```

Use `-Jthreads= -Jramp_sec= -Jloops=` to scale the smoke run.

Required metrics (from your assignment)

For **each** scenario (baseline and stress), collect:

Metric	Where to get it
Average response time	JMeter HTML report / Statistics / Aggregate Report
p95 & p99 response times	JMeter HTML report (Percentiles tables / graphs)
Throughput (requests / second)	JMeter report (Throughput); align with rubric wording
Error rate (%)	JMeter report (Error %); investigate non-200 rows in the JTL if needed
Resource utilization — CPU, memory, DB connections	Not produced by JMeter alone — use EC2 / RDS / container dashboards, htop , Windows Task Manager, pg_stat_activity , or monitoring/postgres-snapshot.sql during/after runs

Regenerate report from JTL without re-running:

```
./bin/jmeter -g results.jtl -o html-report-folder
```

Scenario mapping (course “70% read / 30% write”)

This project has **no** separate REST “read message” API for chat history in the load path. Approximate mapping:

Course idea	This project
“Read”	Mostly GET /health and optional GET /metrics (light)
“Write”	WebSocket TEXT messages (plugin) or attribute “writes” to client_part2 runs and JMeter for HTTP only

[assignment-baseline-100k-5min.jmx](#) (baseline) and [assignment-stress-30min.jmx](#) (stress) implement the graded HTTP scenarios; [assignment-baseline-30min-duration.jmx](#) is an **optional** extra. All map **70/30** with [/metrics](#) **standing in for the “30% write / heavy” slice**; add WebSocket samplers if you need literal chat payloads in JMeter.

State clearly in the PDF how you interpreted **read/write** for grading.

New Assignment Plans (Created)

These two plans were added for your exact requested scenarios:

- `assignment-baseline-1000u-100k-5min-rw7030.jmx`
- `assignment-stress-500u-30min-rw-mixed.jmx`

1) Baseline Performance Test

- Concurrent users: **1000** (700 read + 300 write-mapped)
- Duration: **300s**
- Target calls: **~100K** total via throughput caps
- Mix: **70% read** (`GET /health`) + **30% write-mapped** (`GET /metrics`)

Run:

```
cd "<repo>\load-tests\jmeter"  
.\run-http-via-alb.ps1 -TestPlan assignment-baseline-1000u-100k-5min-rw7030.jmx -  
JMeterInstall "<your-jmeter-root>"
```

2) Stress Test

- Concurrent users: **500** (350 read + 150 write-mapped)
- Duration: **1800s (30 min)**
- Mixed read/write operations
- Default call volume target: **~400K**

Run:

```
cd "<repo>\load-tests\jmeter"  
.\run-http-via-alb.ps1 -TestPlan assignment-stress-500u-30min-rw-mixed.jmx -  
JMeterInstall "<your-jmeter-root>"
```

Call-volume presets for stress (edit Constant Throughput Timer values in the JMX):

- ~200K total in 30 min: read **4667/min**, write **2000/min**
- ~400K total in 30 min (default): read **9333/min**, write **4000/min**
- ~500K total in 30 min: read **11667/min**, write **5000/min**

Required Metrics Collection Checklist

For each scenario, collect from JMeter HTML report (`-e -o ...` output):

- Average response time
- p95 response time
- p99 response time
- Throughput (requests/second)
- Error rate percentage

Resource utilization (CPU, memory, DB connections):

- CPU + memory: collect snapshots during run from `/health` and `/metrics`, or host-level monitoring.
- DB connections: run `monitoring/postgres-snapshot.sql` at start/mid/end of each test and save outputs.

Quick snapshot command (PowerShell):

```
.\monitoring\collect-metrics.ps1 -BaseUrl "http://<alb-dns>"
```